

第1章 汇编语言基础

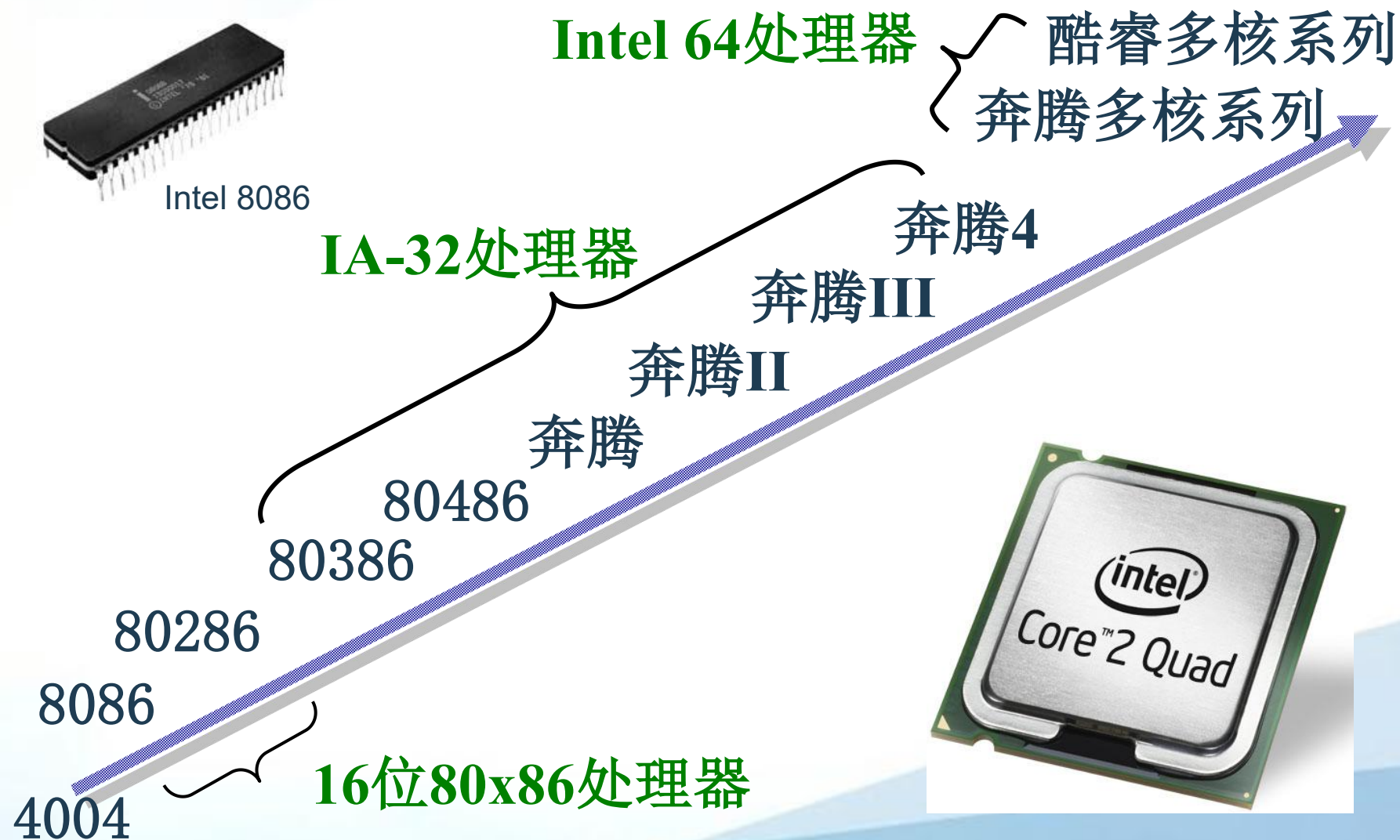
- 1.1 英特尔80x86处理器
- 1.2 个人计算机系统
- 1.3 汇编语言程序格式



第1章 汇编语言基础

- 了解软硬件开发环境
- 熟悉通用寄存器和存储器组织
- 掌握汇编语言的语句格式、程序框架和开发方法

1.1 英特尔80x86处理器



1.1.1 16位80x86处理器

- 16位结构处理器
- 8086/8088指令系统提供16位基本指令集
- 80186/80188增加若干条实用指令
- 8086的工作方式是实方式 (Real Mode)
- 80286增加保护方式 (Protected Mode)
- 80286引入了系统指令
 - 为操作系统等核心程序提供处理器控制功能

指令系统、指令集 (Instruction Set)

1.1.2 IA-32处理器

- 80386引入英特尔32位指令集结构ISA
 - 兼容原16位80286指令系统
 - 全面升级为32位
 - 提供虚拟8086工作方式 (Virtual 8086 Mode)
- 80486集成浮点处理单元支持浮点指令
- Pentium系列
 - 陆续增加若干整数指令、完善浮点指令
 - 增加一系列多媒体指令 (SIMD指令)

IA-32 (Intel Architecture-32)

1.1.3 Intel 64处理器

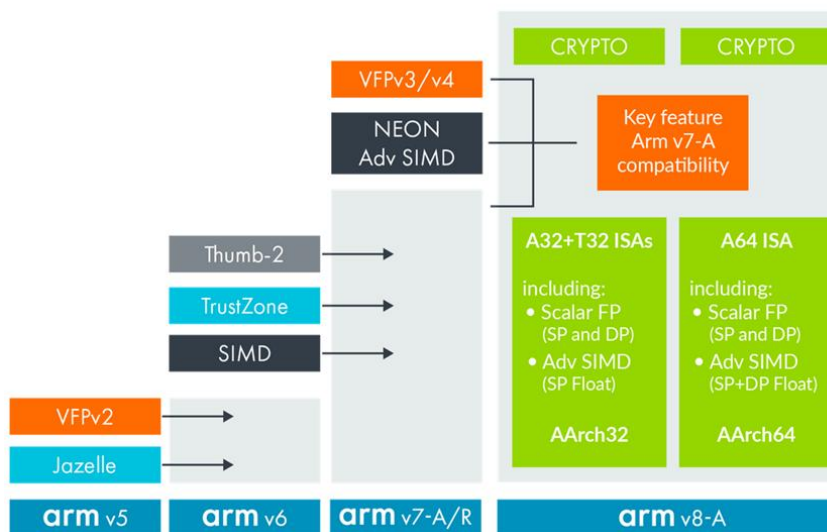
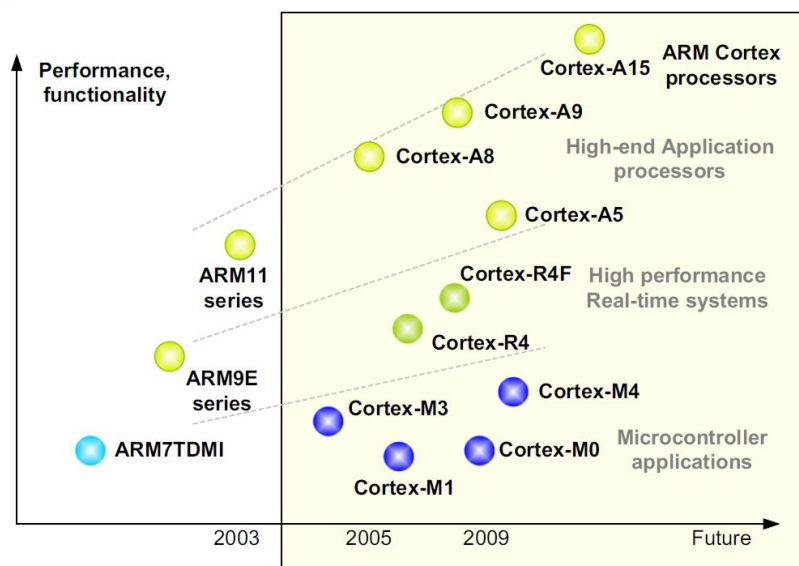
- 引入64位英特尔指令集结构
 - 兼容32位指令系统
 - 新增64位工作方式
- 继续丰富多媒体指令
- 处理器集成多核（Multi-core）技术

Many core



处理器进入多核时代

ARM处理器

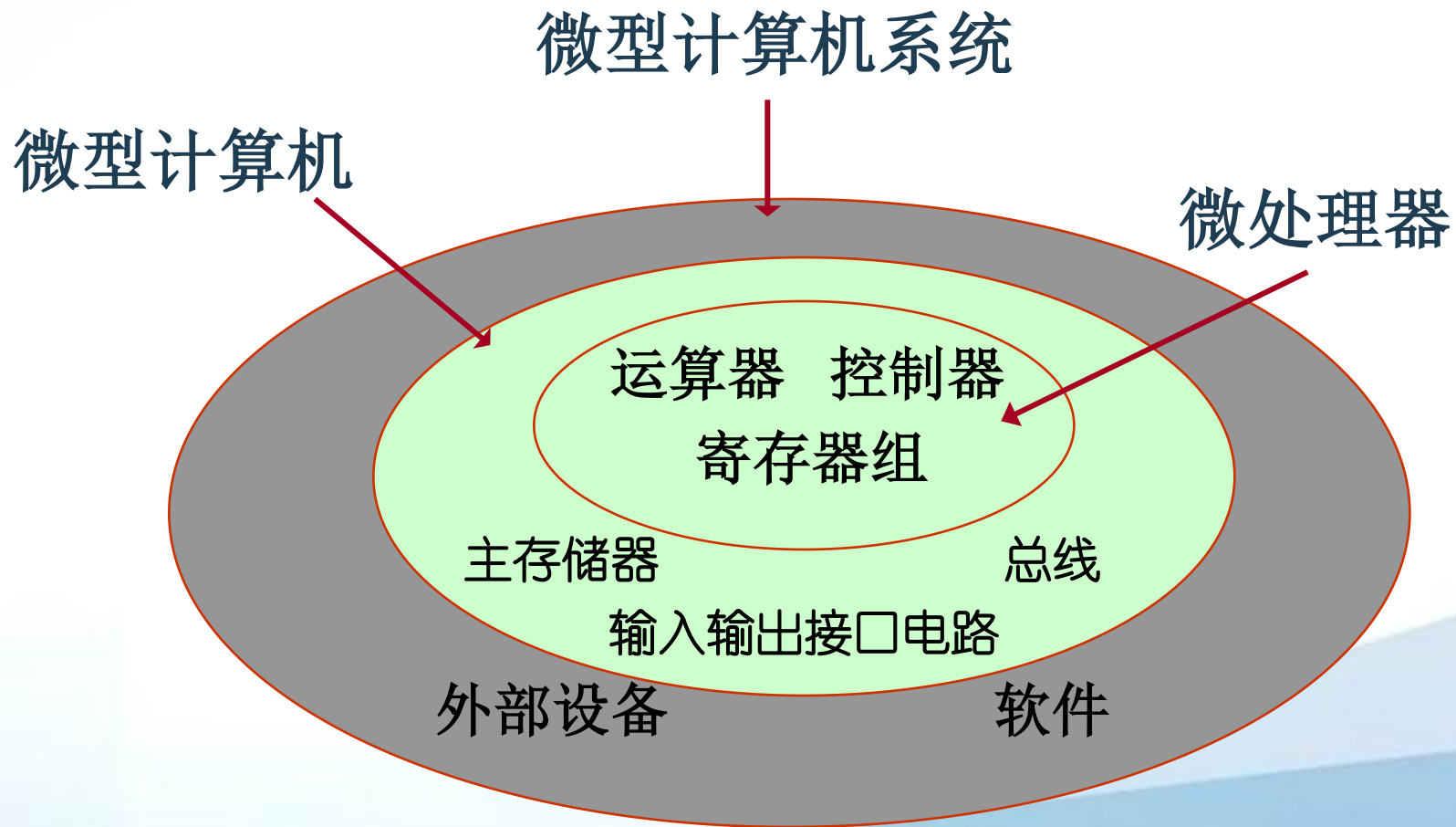


ARM 指令集架构的历史与发展

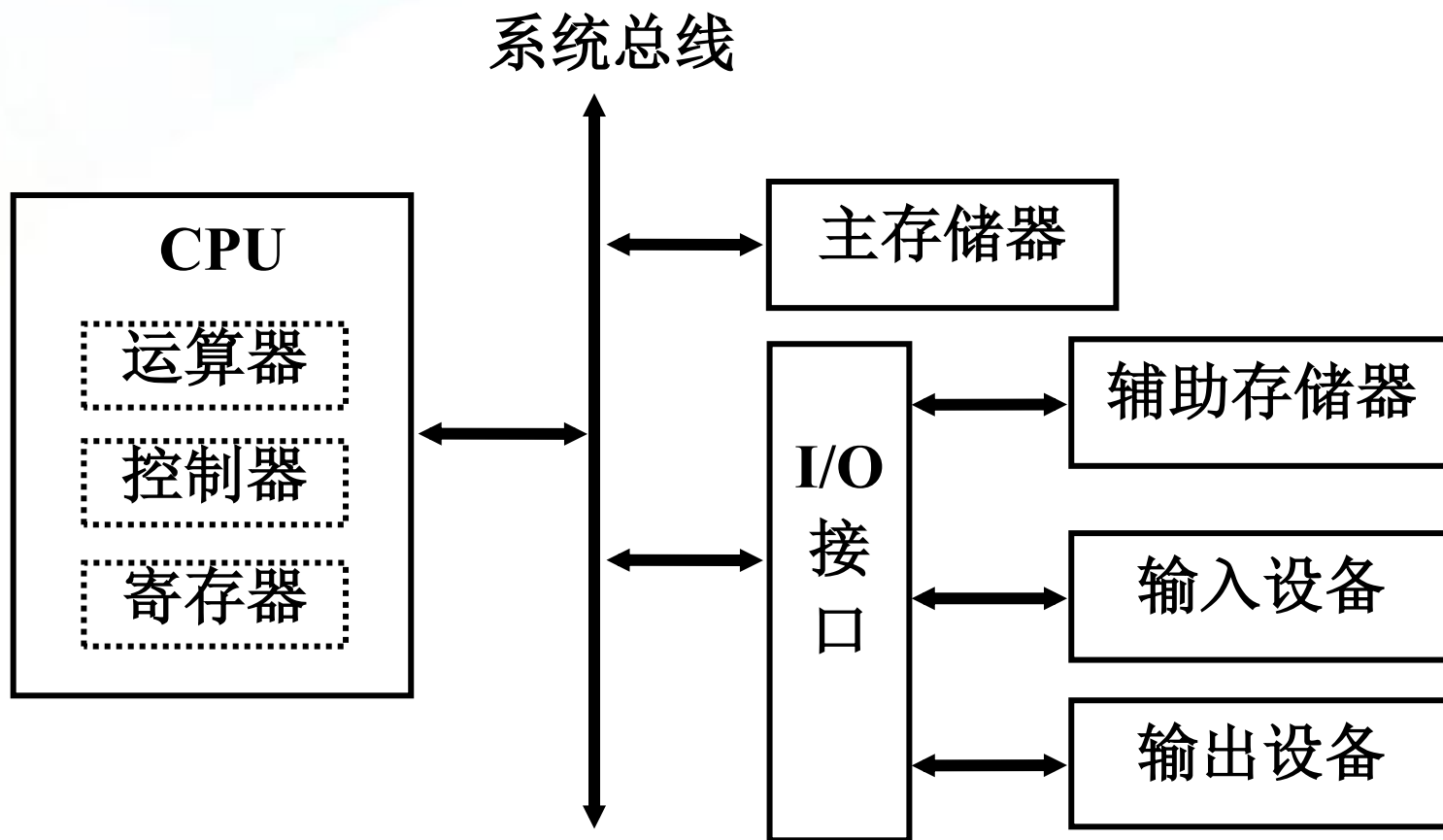
<https://zhuanlan.zhihu.com/p/350962975>

1.2 个人计算机系统

- 硬件（Hardware）：物理设备
- 软件（Software）：程序和文档

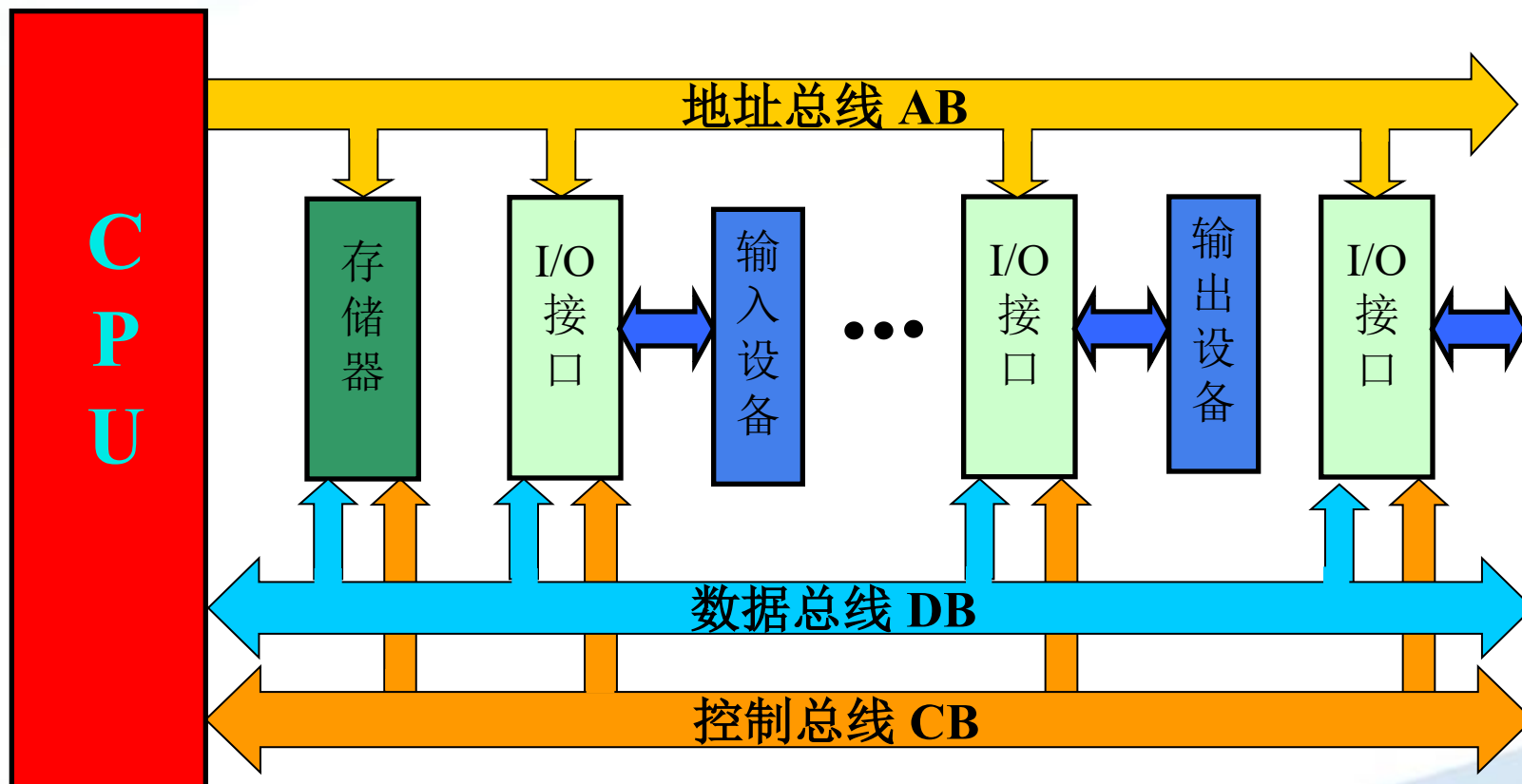


1.2.1 硬件组成



汇编语言程序员将硬件抽象为：
寄存器、存储器地址和输入输出地址

微型计算机的概念结构



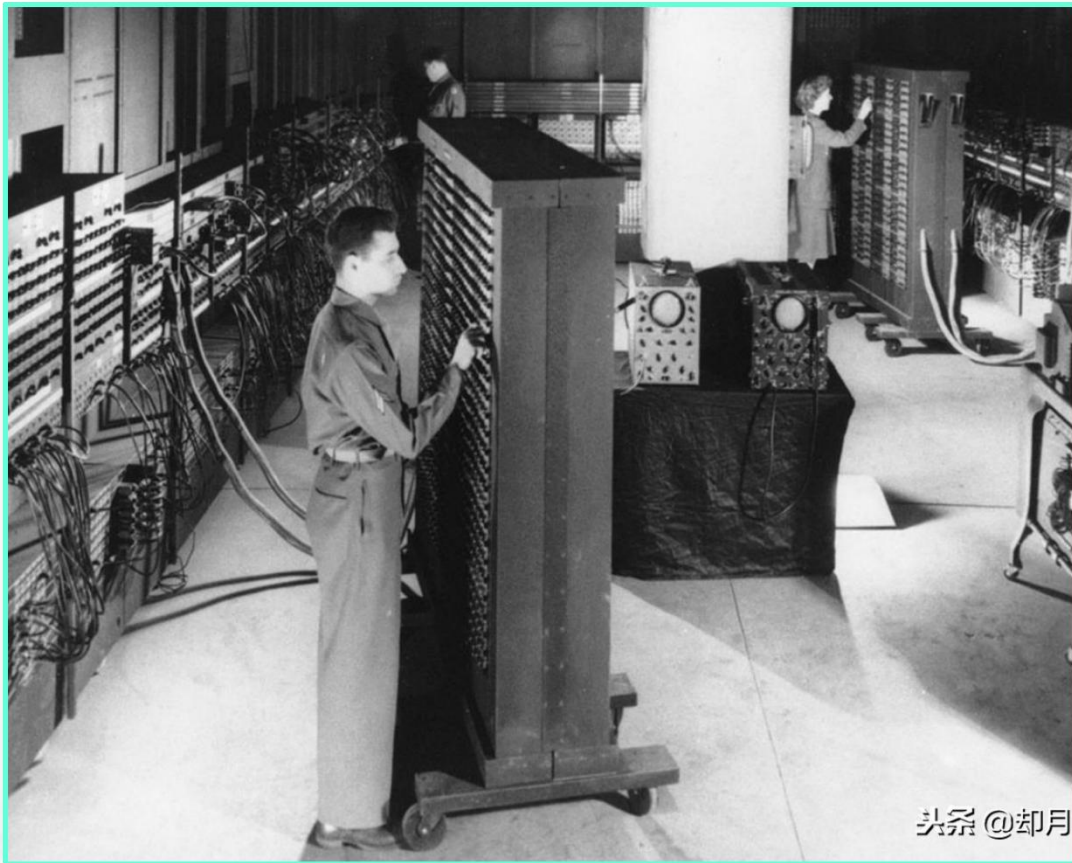
AB: Address Bus

DB: Data Bus

CB: Control Bus

扩展：从机器语言到汇编语言

机器语言：计算机能直接理解的“母语”



核心定义

由0和1组成的二进制代码序列，是CPU唯一能直接识别和执行的语言，无需任何翻译。

指令示例

指令：10110000 01100001

含义：将字符 'a' (ASCII码97) 加载到CPU寄存器中。

主要特点

- 完全硬件相关：依赖特定CPU架构（如Intel/ARM）
- 执行效率最高：直接运行，无中间层开销
- 人类不可读：晦涩难懂，难以直接编写

扩展：从机器语言到汇编语言

机器语言：计算机能直接理解的“母语”



可读性极差

全是0和1的二进制代码，如同天书，难以理解和记忆。



可维护性极低

修改一个小错误可能需要重写大量代码，牵一发而动全身。



硬件依赖性强

与特定CPU架构强绑定，缺乏可移植性，换硬件即失效。



开发效率低下

编写复杂程序耗时费力，极易出错，严重制约软件发展。

扩展：从机器语言到汇编语言

从繁琐到简洁：汇编语言的诞生

机器语言 (Machine Language)

```
10110000 01100001
```

- **晦涩难懂：**由二进制数字组成，人类难以直接阅读和记忆。
- **执行效率：**CPU 可直接执行，是最底层的指令形式。

汇编语言 (Assembly Language)

```
MOV AL, 61h
```

- **助记符：**MOV (Move) 表示数据传送，简单易记。
- **符号化：**用寄存器名(AL)和十六进制(61h)替代二进制。
- **一一对应：**本质是机器语言的“翻译”，架构相关。



核心思想：用人类可读的“助记符”代替晦涩的二进制指令，实现从机器码到符号化编程的跨越。

1.2.2 寄存器 (Register)

- 处理器内部的高速存储单元
- 用于暂时存放程序执行过程中的代码和数据
- 透明寄存器
 - 对应用人员不可见、不能编程直接控制
- 可编程 (Programmable) 寄存器
 - 具有引用名称、供编程使用
 - 通用寄存器 (General-Purpose Register)
 - 具有多种用途
 - 数量较多、使用频度较高
 - 专用寄存器
 - 各自只用于特定目的

扩展：为什么需要寄存器？

寄存器就像是 CPU 的“工作台”或“临时办公桌”，所有的数据运算、地址计算和指令执行都离不开它们。相比计算机的内存（RAM），寄存器的容量非常小（通常每个寄存器只有 32 位或 64 位），但速度却快得多。

扩展：为什么需要寄存器？

寄存器的主要作用

1. 临时数据存储：

寄存器最基本的功能。在进行算术运算（如加法、减法）或逻辑运算（如与、或、非）时，操作数必须先从内存加载到寄存器中，运算结果也会先保存在寄存器里，之后再根据需求写回内存。

示例：要计算 $a + b$ ，需要先将变量 a 和 b 的值分别从内存读取到两个不同的寄存器（如 EAX 和 EBX ），然后执行 `ADD EAX, EBX` 指令，结果就保存在 EAX 寄存器中。

扩展：为什么需要寄存器？

寄存器的主要作用

2. 参与运算与逻辑操作：

CPU 的算术逻辑单元（ALU）只能对存放在寄存器中的数据进行运算。所有复杂的计算最终都会分解为一系列对寄存器数据的简单操作。

扩展：为什么需要寄存器？

寄存器的主要作用

3. 存储内存地址：

寄存器常用来存放内存单元的地址。通过将地址加载到特定的寄存器（如基址寄存器 EBX、变址寄存器 ESI/EDI），可以实现对内存中数据的间接访问。

示例：代码，`mov edx, offset msg` 就是将字符串 `msg` 的内存地址加载到 EDX 寄存器中，然后 `call WriteString` 函数通过 EDX 寄存器的值找到并输出这个字符串。

扩展：为什么需要寄存器？

寄存器的主要作用

4. 控制程序流程：

指令指针寄存器 (EIP)：它存储着 CPU 下一条要执行的指令的内存地址。CPU 会自动读取 EIP 指向的指令并执行，然后 EIP 自动指向下一条指令。跳转指令（如 JMP、JNZ）就是通过修改 EIP 的值来改变程序的执行顺序。

标志寄存器 (EFLAGS)：它包含了一系列状态标志位（如零标志位 ZF、进位标志位 CF、符号标志位 SF 等），这些标志位记录了上一次运算的结果状态（如是否为零、是否有进位、结果是正还是负）。条件跳转指令（如 JE、JC）会根据这些标志位的值来决定是否跳转。

扩展：为什么需要寄存器？

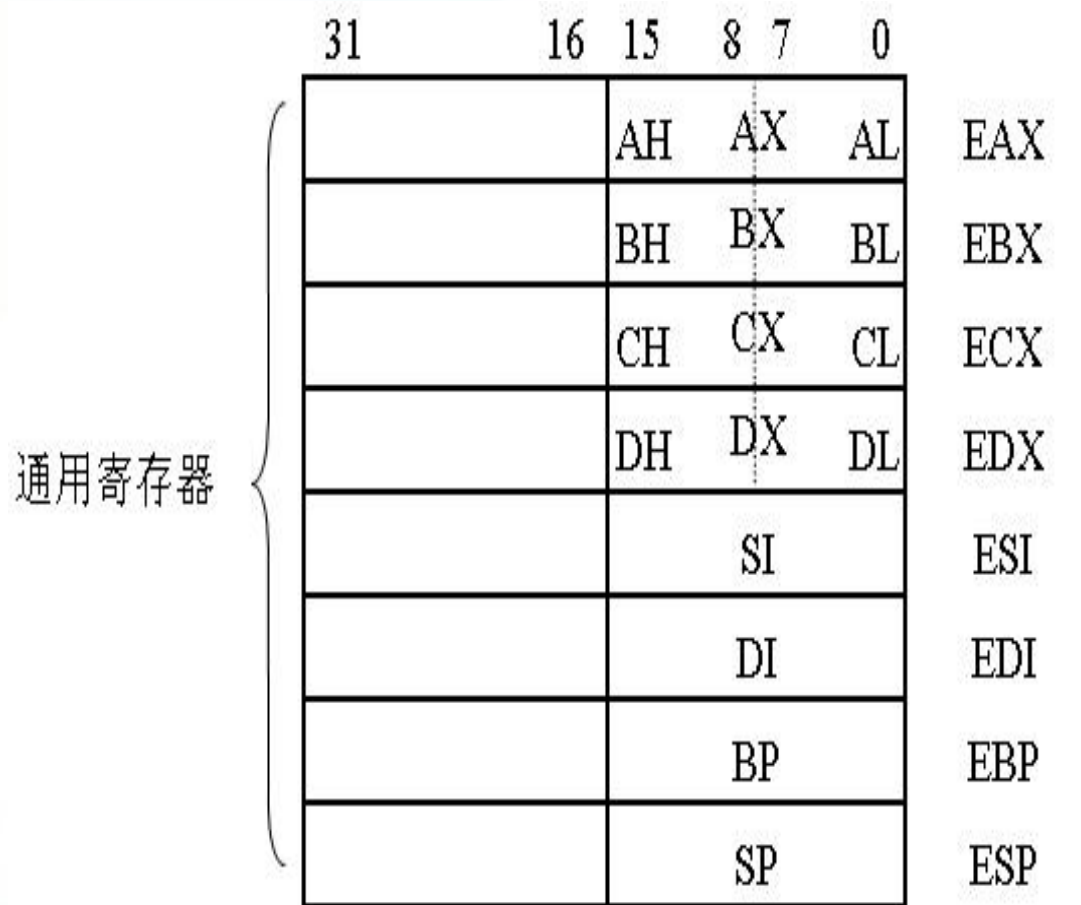
寄存器的主要作用

5. 作为函数调用的桥梁：

在函数调用时，寄存器常被用来传递函数的参数、保存函数的返回地址以及存放函数的返回值。不同的调用约定（如 `stdcall`，`Irvine32` 库默认的调用约定）会规定哪些寄存器用于传递参数，哪些寄存器的值在函数调用后需要被保留。

示例：在 `stdcall` 约定中，函数的返回值通常存放在 `EAX` 寄存器中。

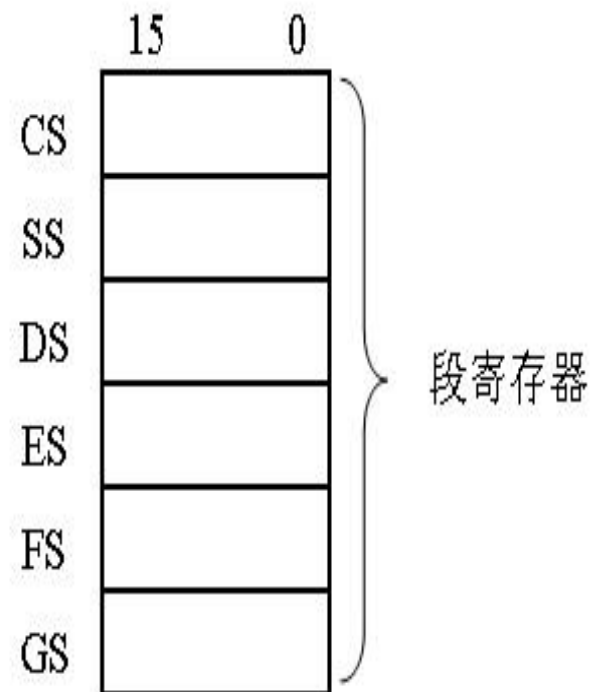
IA-32常用寄存器



标志寄存器

指令指针

	FLAGS	EFLAGS
	IP	EIP

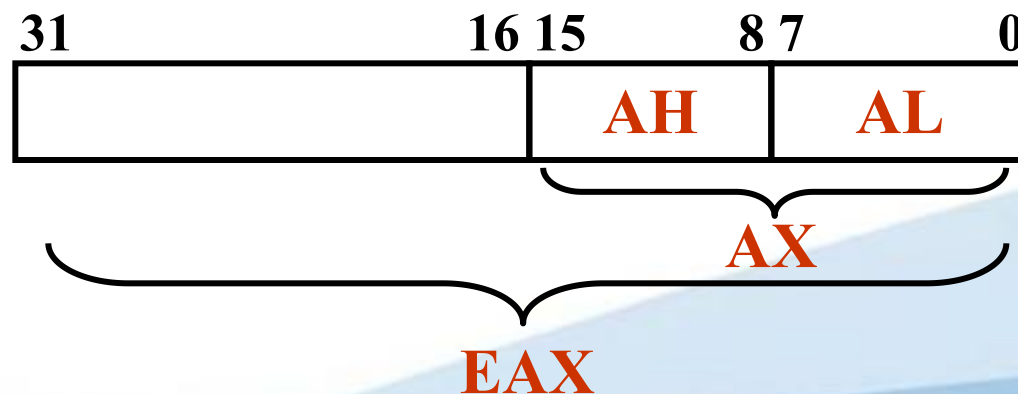


- ✓ 8个32位通用寄存器
- ✓ 6个16位段寄存器
- ✓ 1个32位标志寄存器
- ✓ 1个32位指令指针

1. 通用寄存器

- 处理器最常使用的整数通用寄存器
- 可用于保存整数数据、地址等
- 8个32位通用寄存器
EAX, EBX, ECX, EDX
ESI, EDI, EBP, ESP
- 8个16位通用寄存器
AX, BX, CX, DX
SI, DI, BP, SP
- 8个8位通用寄存器
AH, BH, CH, DH
AL, BL, CL, DL

既是一个整体
又可独立使用



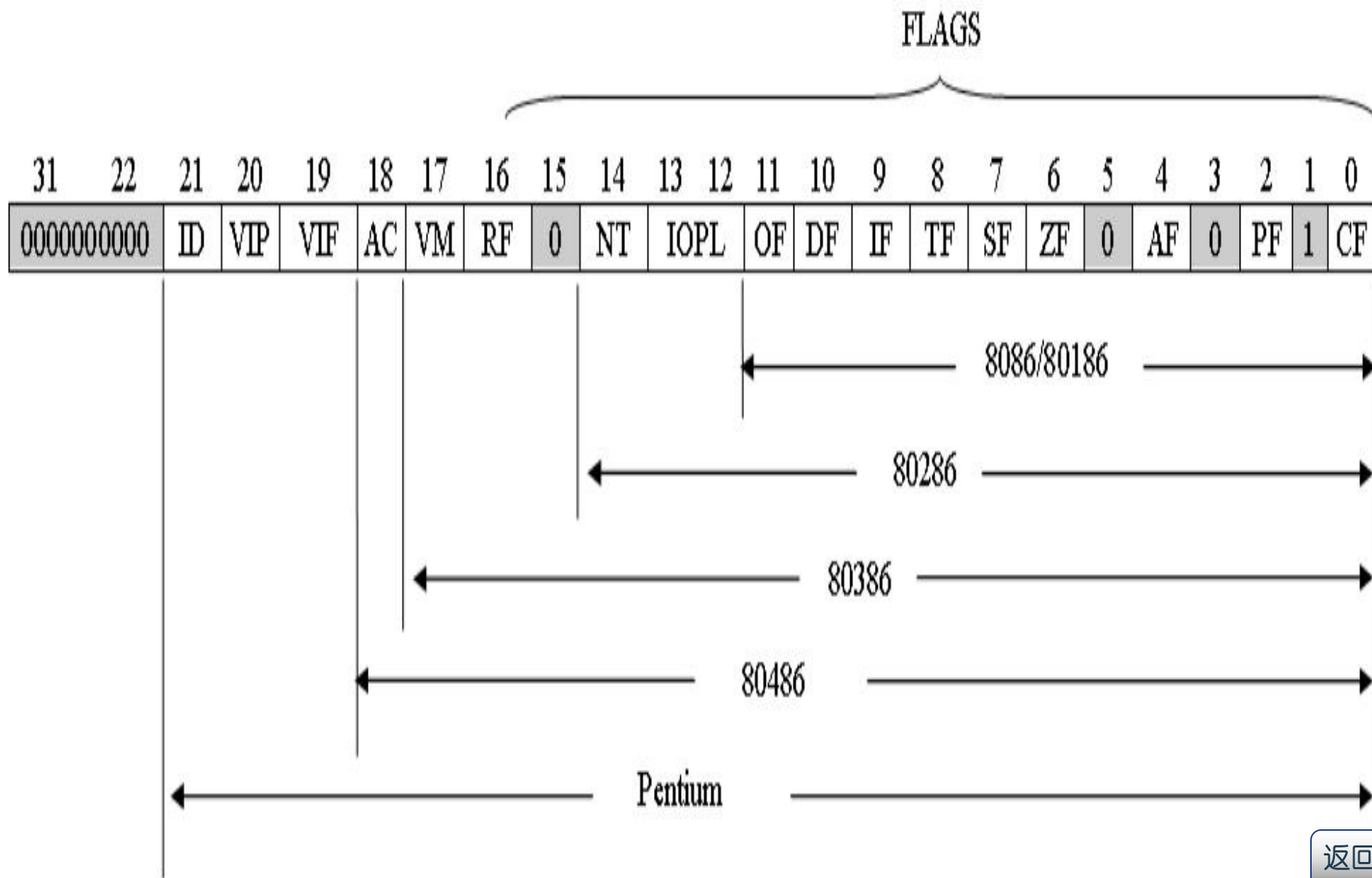
通用寄存器的名称

EAX	Accumulator	累加器
EBX	Base	基址寄存器
ECX	Counter	计数器
EDX	Data	数据寄存器
ESI	Source Index	源变址寄存器
EDI	Destination Index	目的变址寄存器
EBP	Base Pointer	基址指针
ESP	Stack Pointer	堆栈指针

2. 标志寄存器

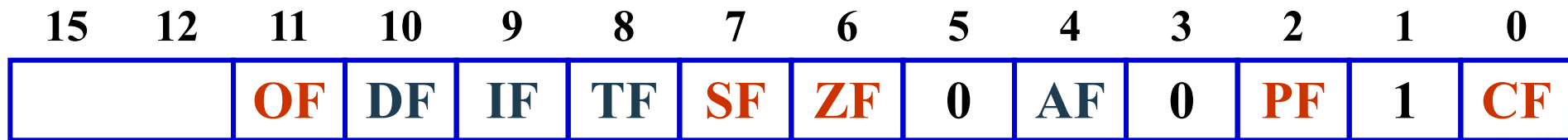
- 标志 (Flag)
 - 反映指令执行结果或控制指令执行形式
 - 用一个或多个二进制位表示一种标志
 - 用0和1的不同组合表达标志的不同状态
- 8086支持16位标志寄存器FLAGS
- IA-32处理器形成32位EFLAGS标志寄存器
 - 状态标志：记录指令执行结果的辅助信息
 - 控制标志：方向标志DF，仅用于串操作指令
 - 系统标志：控制操作系统或核心管理程序的操作方式

标志寄存器EFLAGS



处理器最基本的标志：状态标志

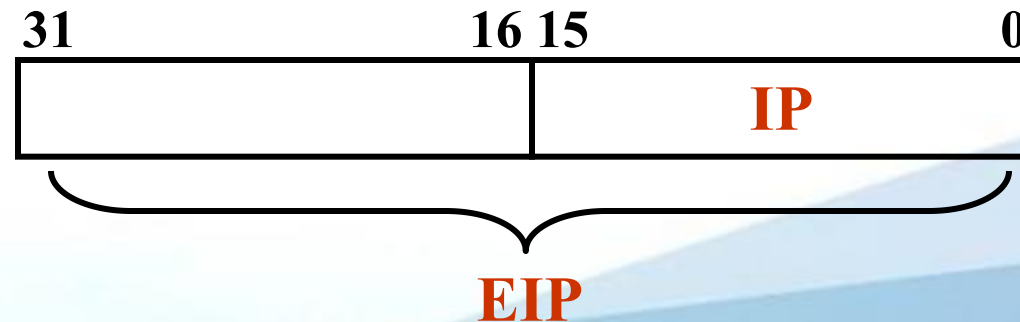
- 用来记录指令执行结果的辅助信息
- 加减运算和逻辑运算指令主要设置它们
- 其他有些指令的执行也会相应地设置它们
- 处理器主要使用其中5个构成各种条件，分支指令判断这些条件实现程序分支



8086的标志

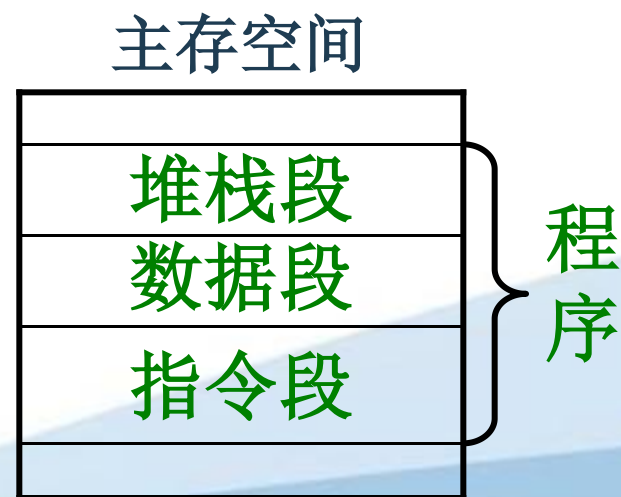
3. 指令指针寄存器EIP

- 保存将要执行的指令在主存的存储器地址
- EIP是专用寄存器
 - 顺序执行时自动增量（加上该指令的字节数），指向下一条指令
 - 分支、调用等操作时执行控制转移指令修改，引起程序转移到指定的指令执行
 - 出现中断或异常时被处理器赋值而相应改变

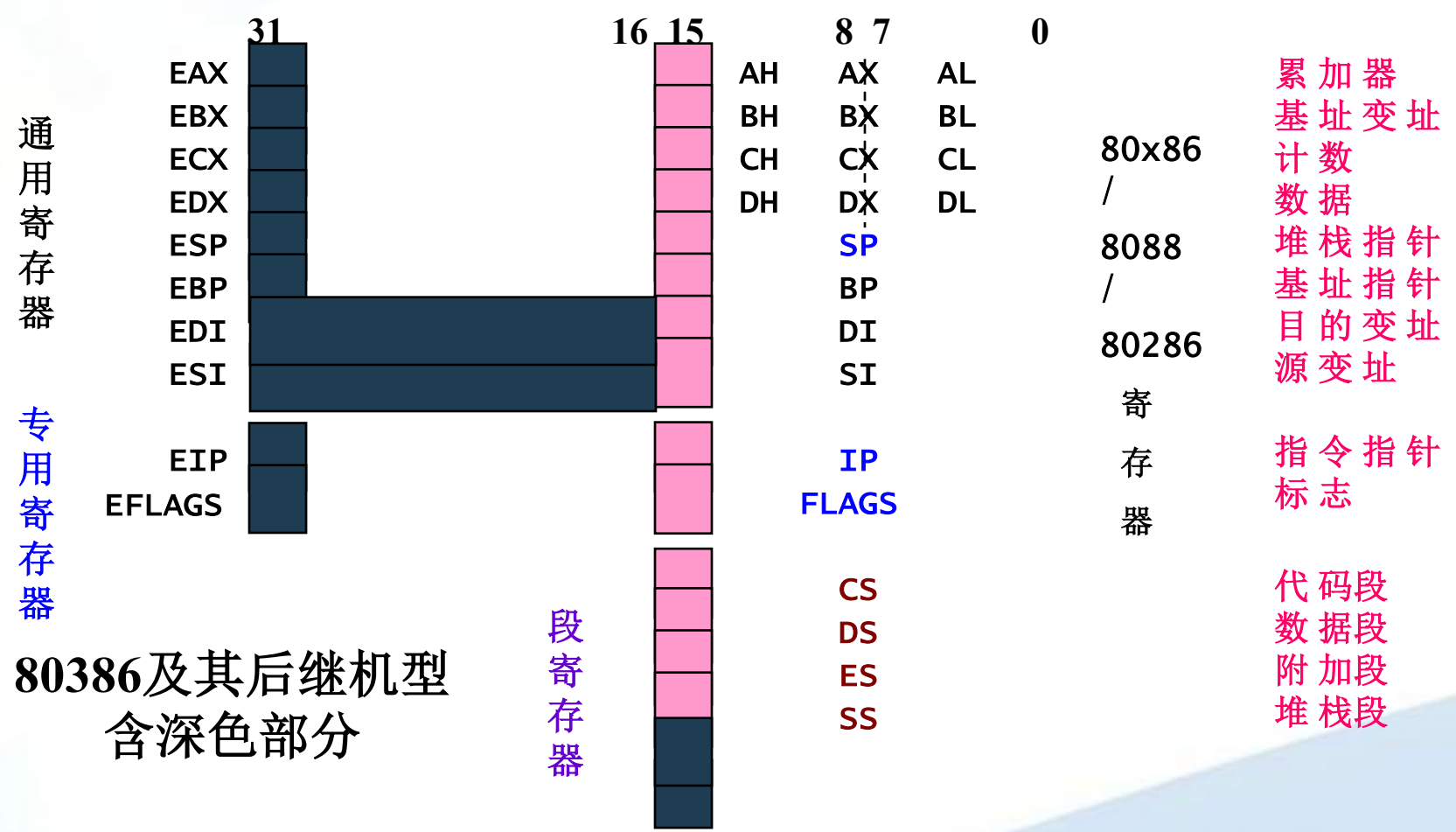


4. 段寄存器

- 段（Segment）是用于安排相关代码或数据的一个主存区域
- 段寄存器表明某个段在主存中的位置
- 6个16位段寄存器：CS DS SS ES FS GS
- 应用程序主要涉及3类段
 - 代码段（Code Segment）
存放程序中指令代码
 - 数据段（Data Segment）
存放当前运行程序所用数据
 - 堆栈段（Stack Segment）
指明程序使用的堆栈区域

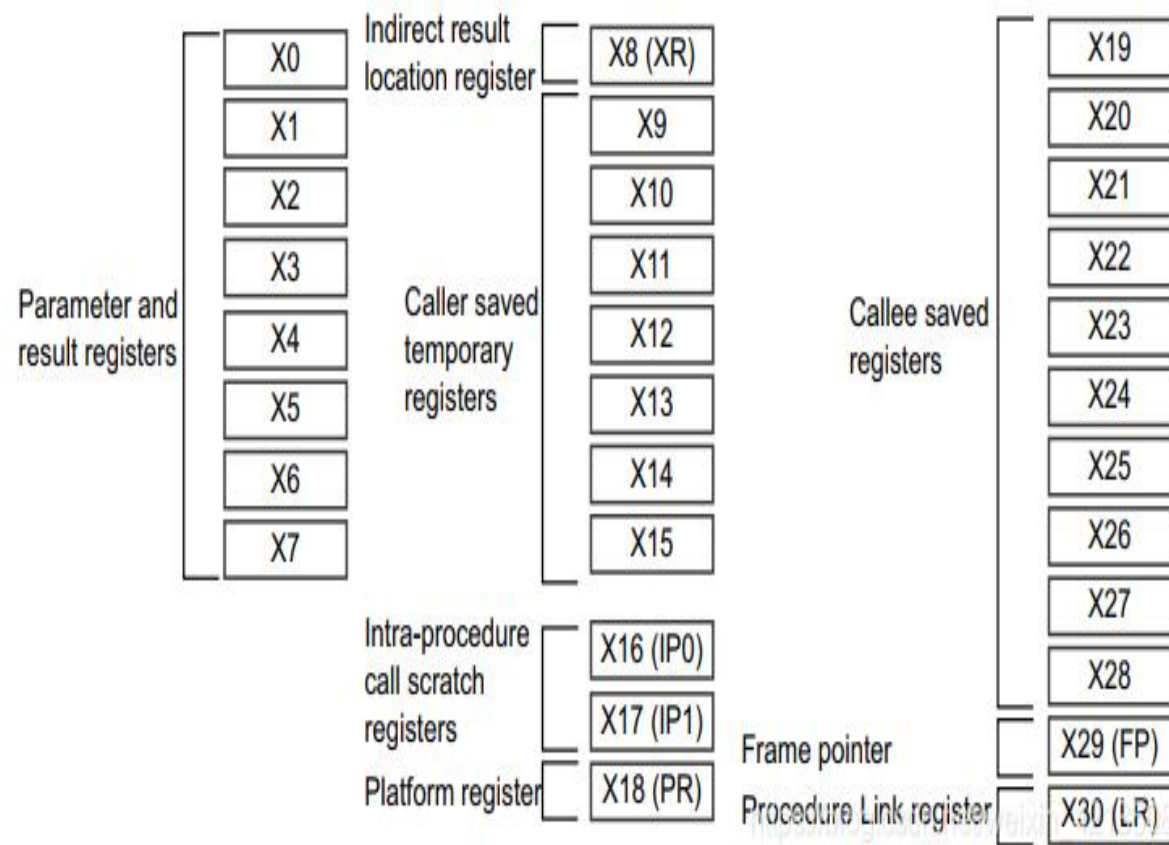


80x86的可见寄存器结构（16个）



80x86的程序可见寄存器组

ARMv8常用寄存器



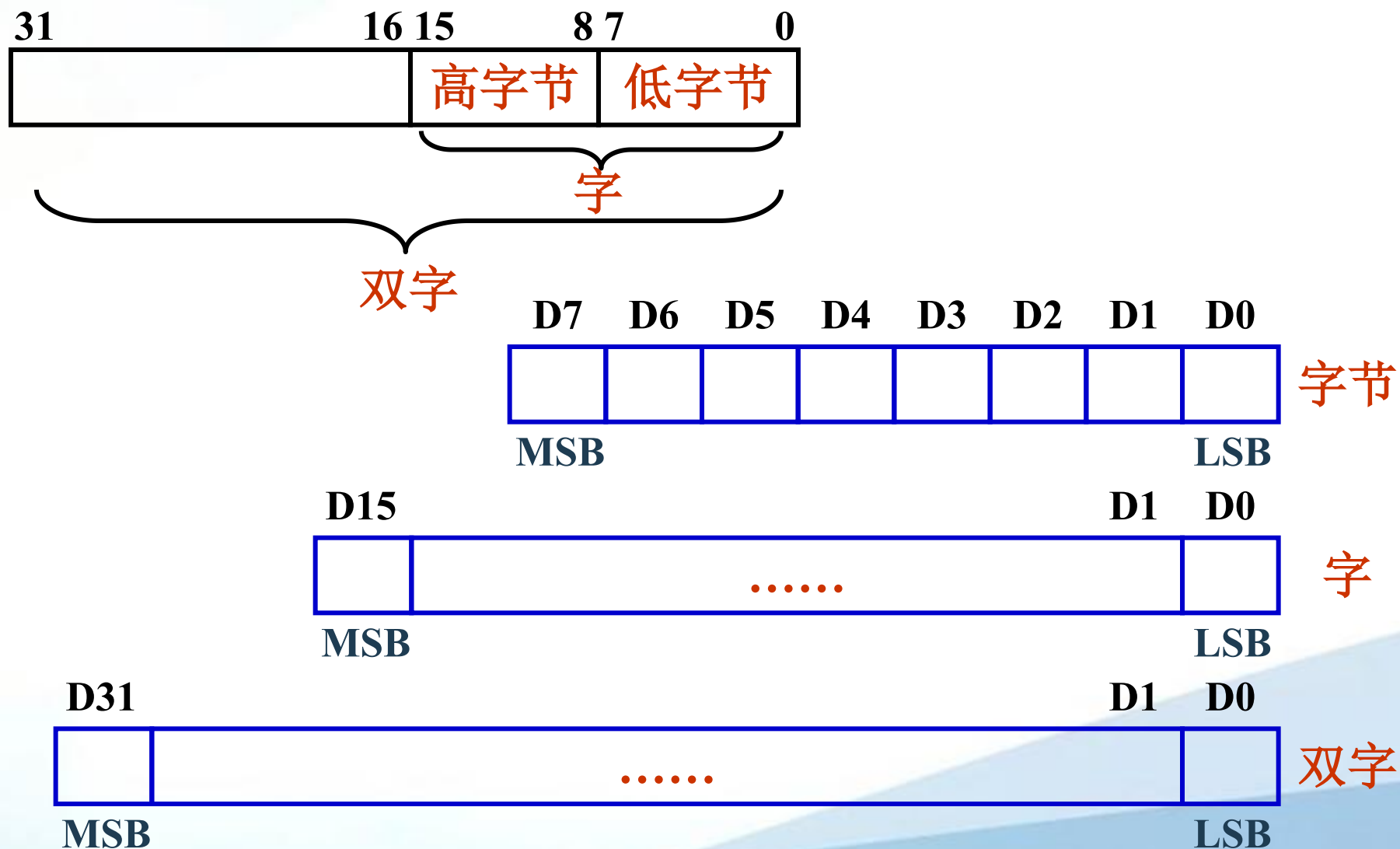
ARMv8-A架构基础之寄存器 <https://www.jianshu.com/p/b29bb8f2e727>

1.2.3 存储器组织

- 主存储器容量很大，被划分成许多存储单元
- 每个存储单元被编排一个号码
 - 即存储单元地址
 - 称为存储器地址 (Memory Address)
- 每个存储单元以字节为基本存储单位
 - 即字节编址 (Byte Addressable)
 - 一个字节 (Byte) 等于8个二进制位 (Bit)
 - 二进制位是计算机存储信息的最小单位
 - 16位 (2个字节) 构成一个字 (Word)
 - 32位 (4个字节) 构成一个双字 (Double Word)

示意图

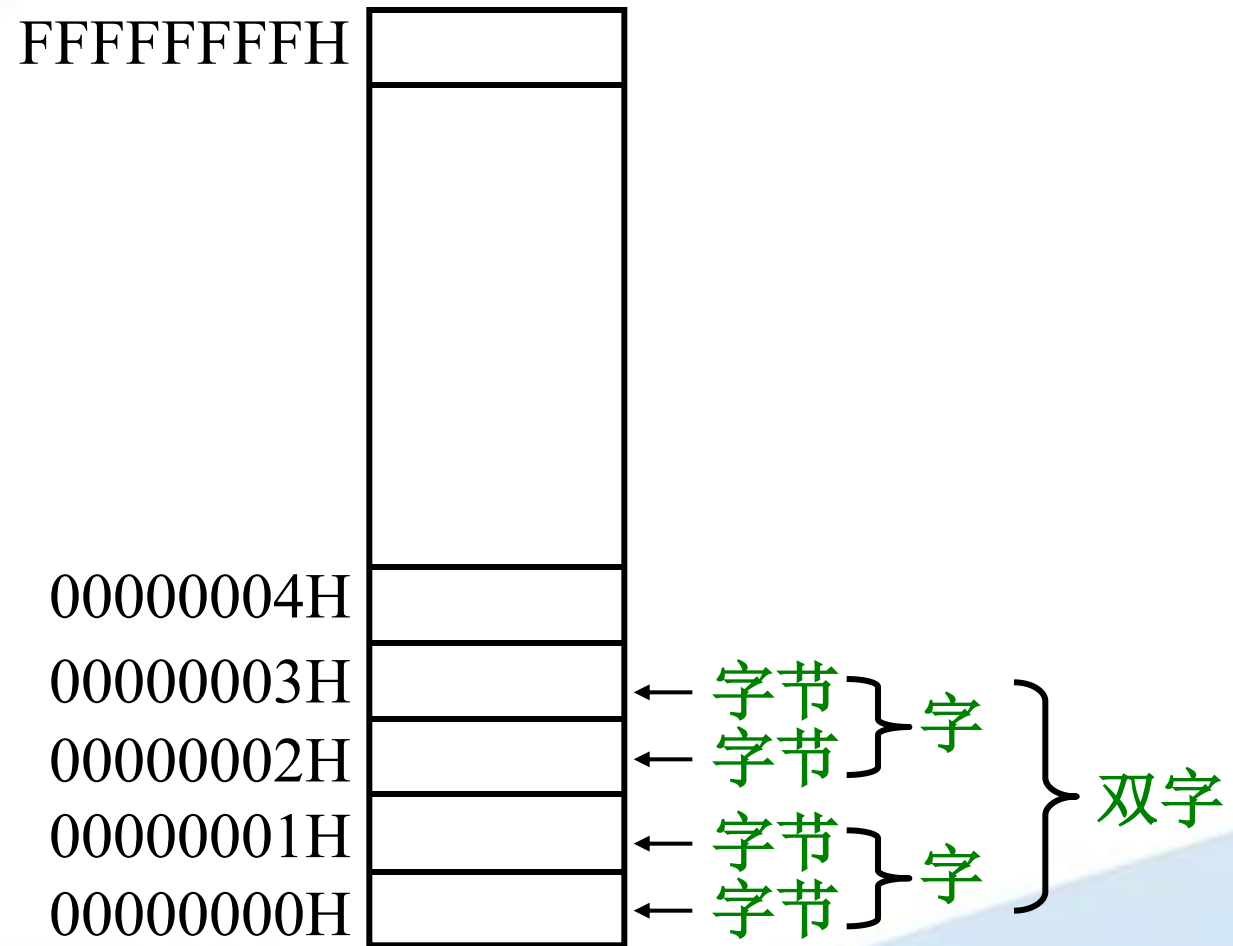
数据的位格式



1. 存储模型

- 物理存储器以字节为基本存储单位
- 每个存储单元被分配一个唯一的地址
- 这个地址就是物理地址
- 物理地址空间从0开始顺序编排，直到处理器支持的最大存储单元
 - 8086处理器支持1MB存储器：00000H~FFFFFH
 - IA-32处理器支持4GB存储器：00000000H~FFFFFFFFH
- 操作系统利用存储管理单元进行存储管理，程序并不直接寻址物理存储器
- IA-32处理器提供3种存储模型，用于程序访问存储器

存储器地址空间



IA-32处理器的存储模型

- (1) 平展存储模型 (Flat Memory Model)
 - 存储器是一个连续的地址空间：线性地址空间
 - IA-32处理器支持4GB容量线性地址空间
- (2) 段式存储模型 (Segmented Memory Model)
 - 存储器由一组独立的地址空间：段 (Segment)
 - 每个段都可以达到4GB容量
 - 在处理器内部，所有的段都被映射到线性地址空间
- (3) 实地址存储模型 (Real-address Memory Model)
 - 8086处理器的存储模型
 - 段式存储模型的特例
 - 线性地址空间最大为1MB容量，段最大为64KB

2. 工作方式

(1) 保护方式 (Protected Mode)

- IA-32处理器固有的工作状态
- 具有强大的段页式存储管理和特权与保护能力
- 使用全部32条地址总线，可寻址4GB物理存储器
- 使用平展或段式存储模型
- 利用虚拟8086方式支持实地址8086软件

(2) 实地址方式 (Real-address Mode)

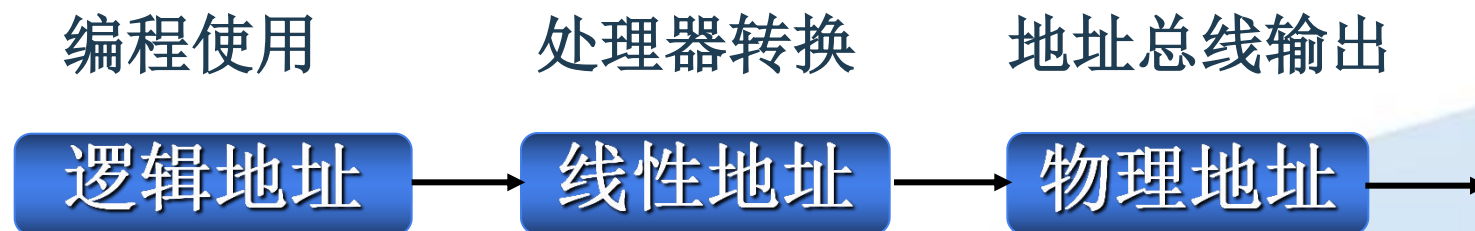
- 可以进行32位处理的快速8086
- 只能寻址1MB物理存储器空间，每个段不超过64KB
- 可以使用32位寄存器、32位操作数和32位寻址方式
- 只能支持实地址存储模型

(3) 系统管理方式 (System Management Mode)

- 实现电源管理和系统安全管理

3. 逻辑地址 (Logical Address)

- 在处理器内部、程序员编程时采用的地址
- 逻辑地址 = 段基地址 : 偏移地址
 - 段基地址 = 在主存中的起始地址
 - 偏移地址 = 距离段基地址的位移量
- 某个存储单元可以有多个逻辑地址，但只有一个唯一的物理地址



示意图

扩展：一个核心问题：为什么要“分段”？

运行的基石

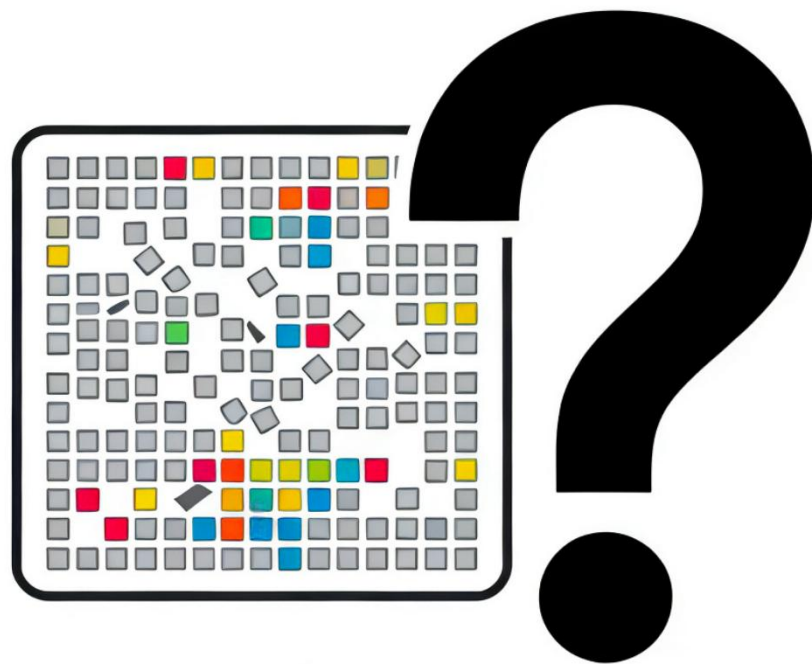
程序和数据必须加载到内存中运行。为何不将内存视为连续的“地址池”直接访问，而非要划分代码、数据、堆栈等区域？

划分的逻辑

这种人为的划分看似增加了复杂度，实则是程序逻辑清晰化与内存管理安全性的内在需求。

背后的答案

这背后既有**历史的必然性**，也有现代操作系统对**高效管理**的追求。



扩展：突破地址空间的物理限制

🔧 硬件矛盾：Intel 8086 的困境

- 寄存器限制：内部16位寄存器，寻址范围仅 $2^{16} = 64\text{KB}$ 。
- 物理潜力：外部20条地址线，理论可访问 $2^{20} = 1\text{MB}$ 内存。
- 核心问题：16位寄存器无法直接生成20位物理地址。

🔧 解决方案：分段地址合成

- 内存分段：将1MB空间划分为多个64KB的“段”。
- 合成公式：物理地址 = (段地址 $\ll 4$) + 偏移地址。
- 效果：通过两个16位寄存器的配合，实现20位寻址。



扩展：程序逻辑：代码、数据、栈的天然分离



代码段/指令段 (Code Segment)

存放CPU执行的指令序列，通常为只读，防止运行时被意外修改。



数据段 (Data Segment)

存放常量、全局变量及静态变量，内容为可读写。



堆栈段 (Stack Segment)

用于函数调用、参数传递及局部变量存储，遵循后进先出 (LIFO) 原则。



扩展：分段的优势：实现内存保护与隔离



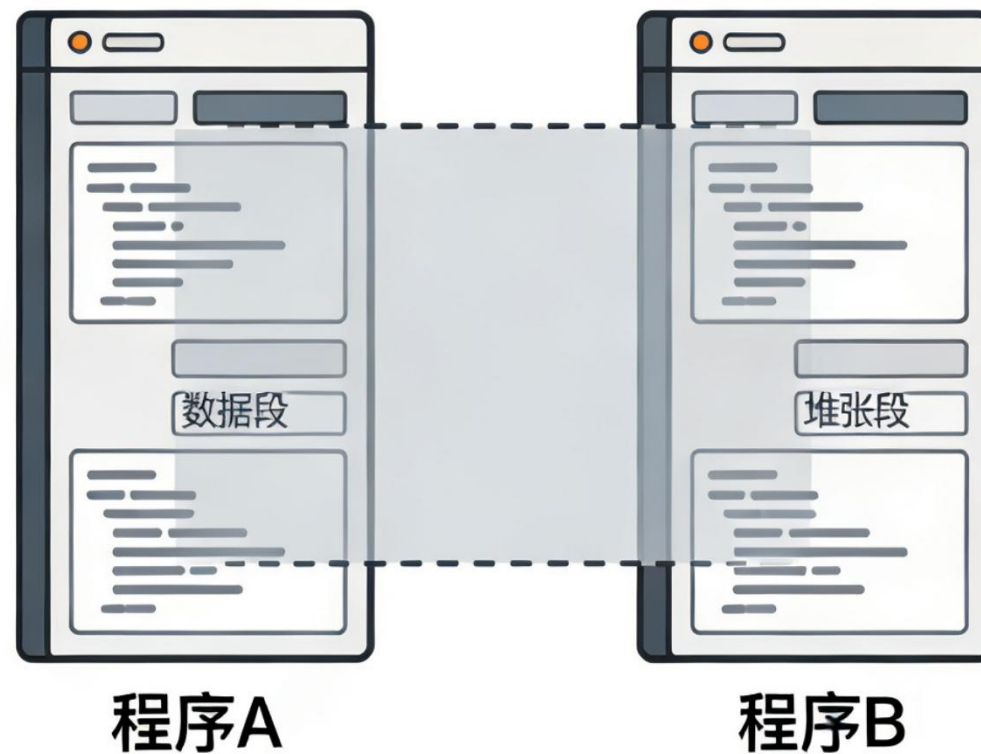
隔离不同程序：系统级安全屏障

在多任务环境下，每个程序被分配独立内存段。操作系统利用段描述符机制，严格限制程序只能访问自身地址空间，防止一个程序的错误操作破坏其他程序或系统内核，确保系统整体稳定性。



隔离程序内部：代码与数据分离

对程序内部的代码段、数据段设置差异化权限。例如将代码段设为只读，防止指令被意外篡改；数据段设为可读写，既保证了程序逻辑的完整性，又增强了自身的健壮性。



逻辑地址与物理地址

逻辑地址 = 相对地址: **205** (2层05号房间)

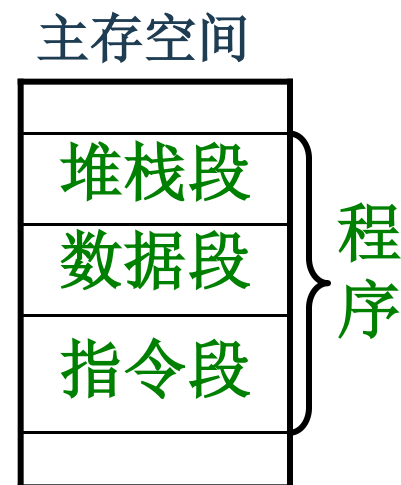
301	302	303	304	305	306	307	308	309	310
201	202	203	204	205	206	207	208	209	210
101	102	103	104	105	106	107	108	109	110

物理地址 = 绝对地址: **15** (第15号房间)

21	22	23	24	25	26	27	28	29	30
11	12	13	14	15	16	17	18	19	20
01	02	03	04	05	06	07	08	09	10

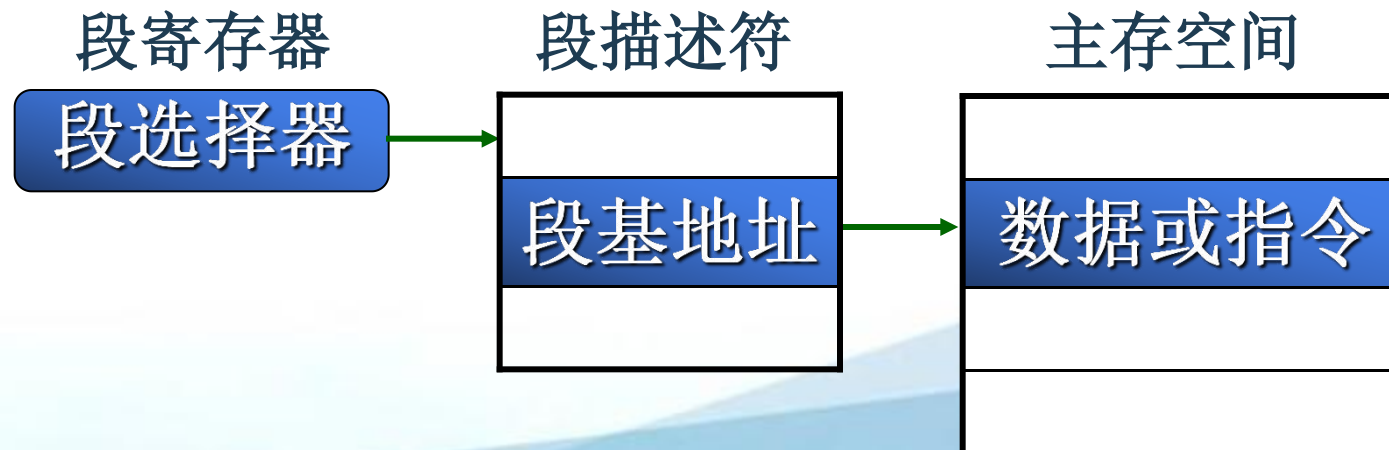
基本段的逻辑地址

- 代码段 (Code Segment)
 - 段基地址: 代码段寄存器CS指示
 - 偏移地址: 指令指针寄存器EIP保存
- 数据段 (Data Segment)
 - 段基地址: 数据段寄存器DS指示
 - 有时也用附加段寄存器ES, 段寄存器FS和GS指示
 - 偏移地址: 各种存储器寻址方式计算出来
- 堆栈段 (Stack Segment)
 - 段基地址: 堆栈段寄存器SS指示
 - 偏移地址: 堆栈指针寄存器ESP保存

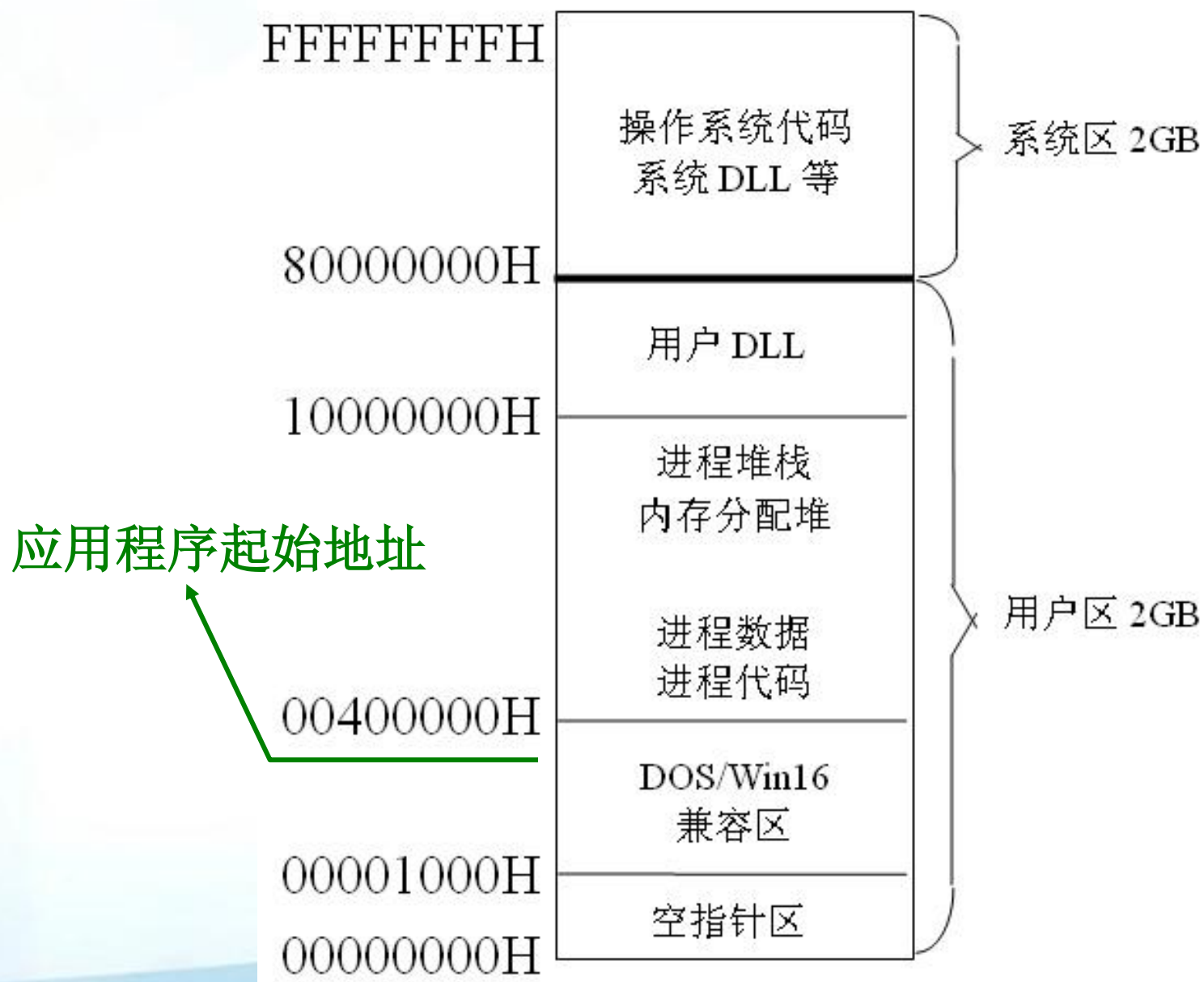


4. 段选择器

- 16位段寄存器保存16位段选择器
- 段选择器指向64位段描述符 (Descriptor)
- 段描述符包括段基地址
 - 平展存储模型：指向地址0位置
 - 段式存储模型：指向线性地址空间不同的段
 - 实地址存储模型：保存段基地址的高16位



5. Win32的虚拟地址分配



1.2.4 程序设计语言

- 机器语言 (Machine Language)
 - 底层的计算机语言，对应机器指令
 - 形成目标（代码）程序
- 汇编语言 (Assembly Language)
 - 将机器指令用助记符号代替而形成的一种语言
 - 本质上是机器语言
 - 需要汇编程序“**汇编**”（翻译）
- 高级语言 (High Level Language)
 - 面向问题的程序设计语言
 - 需要“**编译**”或“**解释**”后执行

指令是处理器的母语

```
mov eax,100  
add eax,256
```

Instructions: Language of the Machine

To command a computer's hardware, you must speak its language. The words of a machine's language are called instructions, and its vocabulary is called an instruction set.

为了控制计算机硬件，必须使用它的语言。机器语言的单词称为指令，它的词汇表称为指令集。

“Computer Organization & Design

-- The Hardware/Software Interface (2nd Edition)”

by John L. Hennessy and David A. Patterson

Morgan Kaufmann Publishers, Inc. 1998

汇编语言和高级语言的比较

- 汇编语言与处理器密切相关，可移植性较差
 - 高级语言与具体计算机无关，便于移植
- 汇编语言功能有限、涉及硬件细节，编写繁琐困难
 - 高级语言功能强大，语法自然，容易掌握和应用
- 汇编语言的优势：直接、有效地控制硬件，容易产生运行速度快、指令序列短小的高效率目标程序
 - 高级语言不易直接控制计算机的各种操作，编译程序产生的目标程序往往比较庞大

混合编程、取长补短

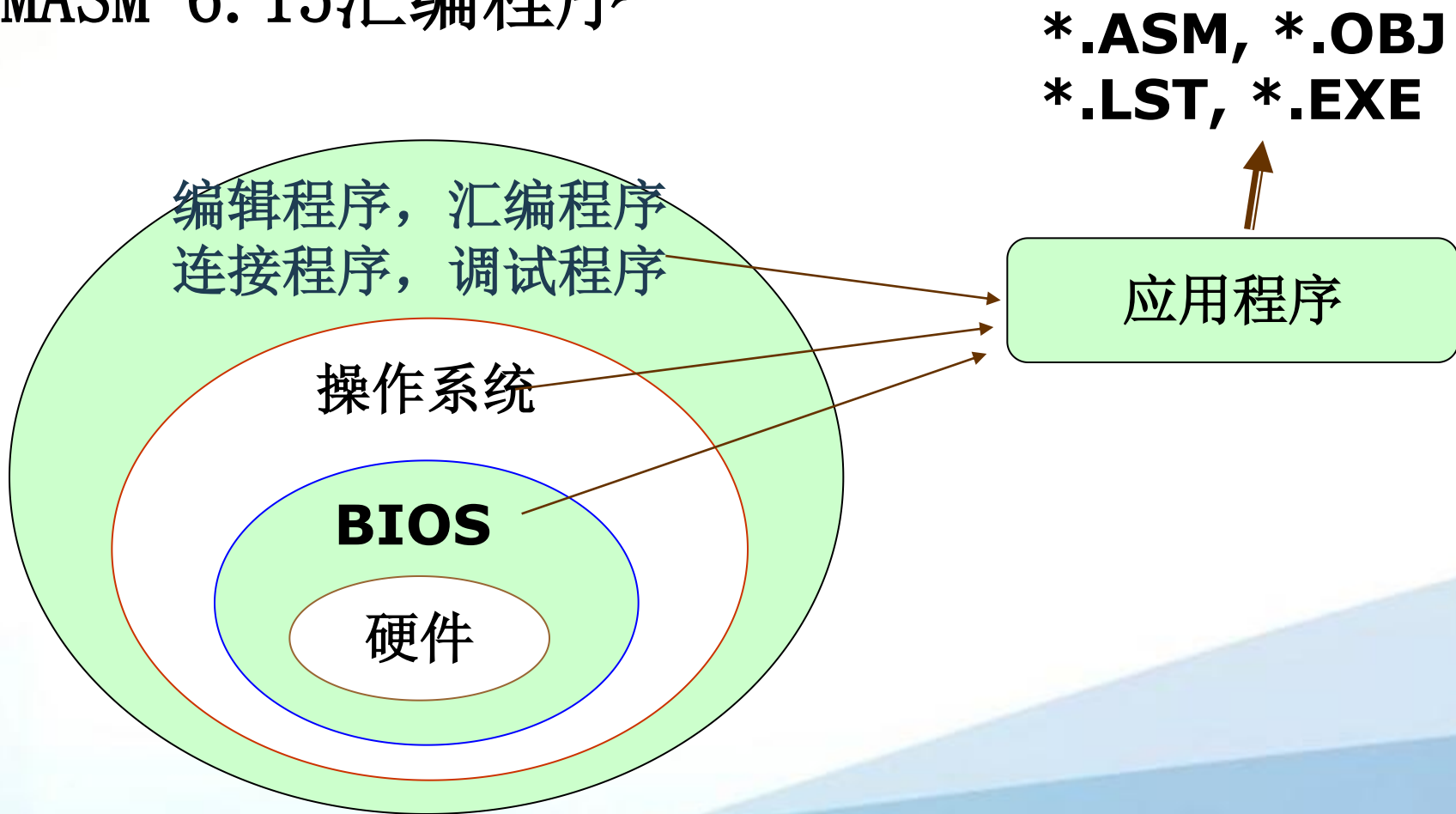


汇编语言的主要应用场合

- 程序要具有较快的执行时间，或者只能占用较小的存储容量
- 程序与计算机硬件密切相关，程序要直接、有效地控制硬件
- 大型软件需要提高性能、优化处理的部分
- 没有合适的高级语言、或只能采用汇编语言的时候
- 分析具体系统尤其是该系统的低层软件、加密解密软件、分析和防治计算机病毒等等

1.2.5 软件系统

- Windows操作系统平台
- MASM 6.15汇编程序



1. 操作系统 (Operating System)

- 操作系统管理着系统的软硬件资源
 - 为用户提供使用机器的交互界面
 - 为程序员使用资源提供可供调用的驱动程序
 - 为其他程序构建稳定的运行平台
- 32位控制台环境
`%SystemRoot%\system32\cmd.exe`
- 16位模拟DOS环境
`%SystemRoot%\system32\command.com`

32位控制台环境和16位模拟DOS环境
两种基本功能和界面一致，但本质不同

2. 汇编程序 (Assembler)

- 将汇编语言源程序转换为机器代码
- 微软宏汇编程序MASM
 - MASM 6.11是最后一个独立发行的MASM软件包
 - MASM 6.15来自Visual C++6.0，支持到Pentium 4的SSE2指令系统
 - Visual C++.NET 2005提供
 - 支持SSE3指令的MASM 8.0
 - 支持64位指令的ML64.EXE程序



汇编语言程序与汇编程序
是两个不同的概念

3. 文件路径

- 目录 (Directory) : 管理磁盘上文件的文件夹
- 路径 (Path) : 分区:\根目录\子目录\文件名
 - 绝对路径: 文件的完整路径
`d:\masm\progs\eg0101.asm`
 - 相对路径: 相对当前目录的路径
`progs\eg0101.asm`
`..\bin\ml.exe`
- 转换分区
`d:`
- 进入当前目录 (Current Directory)
`cd \masm`

4. 内部命令和外部命令

- 内部命令：32位控制台或DOS环境本身具有的、直接支持的命令，即驻留主存的命令
 - 使用方法：命令关键字 参数选项
- 外部命令：与其他可执行文件一样，以文件形式保存在磁盘上的命令
 - 执行方法：路径\文件名 参数
- 路径搜索顺序
 - 先当前目录
 - 后搜索路径
 - 内部命令PATH查看和设置当前的搜索路径



你所执行的命令
不是内部或外部命令，
也不是可运行的程序或批处理文件

5. 进入MASM目录的批处理文件

- 为便于进入MASM目录，可创建批处理文件
 - 32位Windows控制台：WIN32.BAT
 - 16位DOS环境：DOS16.BAT
- 为便于访问其中文件，增加搜索路径：

```
@set PATH=D:\MASM;D:\MASM\BIN;%PATH%
```

在Windows资源管理器打开文件夹“D:\MASM”，
双击该批处理文件，就可以进入MASM环境

具有搜索路径的批处理文件

WIN32.BAT

```
@echo off
@set PATH=D:\MASM;D:\MASM\BIN;%PATH%
%SystemRoot%\system32\cmd.exe
@echo on
```

DOS16.BAT

```
@echo off
@set PATH=D:\MASM;D:\MASM\BIN;%PATH%
%SystemRoot%\system32\command.com
@echo on
```


1.3 汇编语言程序格式

- 程序用程序设计语言编写，由指令构成
- 指令由操作码和操作数（地址码）组成
- 操作码（Opcode）表明处理器执行的操作
 - 例如数据传送、加法运算、跳转等操作。
- 操作数（Operand）是参与操作的数据对象
 - 主要以寄存器名或地址形式指明数据的来源
 - 通常的指令都有一个或两个操作数
 - 有些指令无操作数，个别指令有3或4个操作数
 - 多数操作数需要显式指明，有些操作数隐含使用

1.3.1 指令的代码格式 (Instruction Format)

- IA-32处理器采用可变长度指令格式
- 操作码
 - 可选的指令前缀（用于扩展指令功能）
 - 1~3字节的主要操作码
- 操作数
 - 可选的寻址方式域（包括ModR/M和SIB字段）
 - 可选的位移量
 - 可选的立即数

0~4 字节

1~3 字节

0 或 1 字节

0 或 1 字节

0、1、2 或 4 字节

0、1、2 或 4 字节

指令前缀	操作码	ModR/M	SIB	位移量	立即数
------	-----	--------	-----	-----	-----

使用最多、最基本的数据传送指令

- 传送指令的助记符: MOV (取自Move)

功能演示

- 将数据从一个位置传送到另一个位置
- 类似高级语言的赋值语句

mov dest, src

;源操作数src: 被传送的数据或数据所在的位置

;目的操作数dest: 数据将要传送到的位置

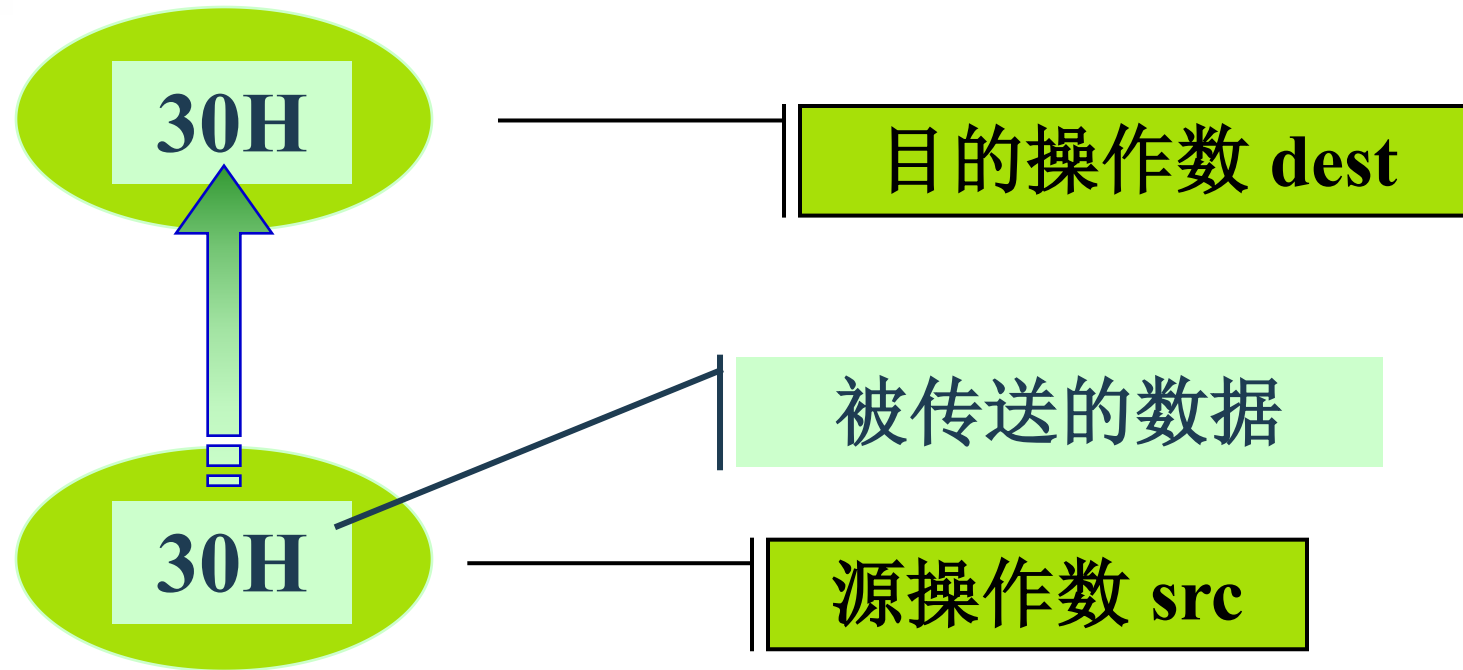
mov eax, ebx ; 机器代码: **8B C3**

mov eax, [ebx] ; 机器代码: **8B 03**

mov eax, [ebx+esi*4+80h]

; 机器代码: **8B 84 B3 80 00 00 00**

传送指令MOV的功能演示



1.3.2 语句格式

- 源程序由语句组成
- 一个语句常占一行（MASM有续行符“\”）
- 一个语句不超过132个字符，4个部分
- **执行性语句：**表达处理器指令（硬指令）
标号：硬指令助记符 操作数, 操作数 ;注释
- **说明性语句：**表达伪指令，控制汇编方式
名字 伪指令助记符 参数, 参数, …… ;注释

这是MASM语法，但具有一般性

1. 标号与名字

- 标号：执行性语句中
 - 冒号分隔
 - 表示处理器指令在主存中的逻辑地址
 - 指示分支、循环等程序的目地址
- 名字：说明性语句中
 - 空格或制表符分隔
 - 变量名、段名、子程序名等
 - 反映变量、段和子程序等的逻辑地址

标号和名字是用户自定义的标识符

标识符 (Identifier)

- 最多由31个字母、数字及规定的特殊符号（如 _、\$、?、@）组成，**不能以数字开头**
- 一个源程序中，用户定义的每个标识符**必须唯一**
- **不能是**汇编程序采用的**保留字**
- 保留字 (Reserved Word) 是编程语言本身需要使用的各种具有特定含义的标识符、也称为关键字
 - 硬指令助记符: MOV
 - 伪指令助记符: BYTE
 - 操作符: OFFSET
 - 寄存器名: EAX

取名原则类似高级语言
但默认不区别大小写字母

2. 助记符

- 助记符是帮助记忆指令功能的符号
 - 硬指令助记符表示处理器指令
 - 伪指令助记符表达一个汇编命令
- 处理器指令示例：传送指令 **MOV**
- 伪指令示例：字节变量定义
 - 助记符： **BYTE** （或**DB**）
 - 功能：在主存中占用若干的存储空间，用于保存变量值，该变量以字节为单位存取

```
msg byte 'Hello, Assembly !',13,10,0
```

2. 助记符

- 组合13, 10（即 0Dh, 0Ah），在 Windows 系统中表示一个回车换行操作。这是控制台输出中最常用的换行方式，效果就是：
- 先将光标移到当前行的开头（13）。
- 再将光标移到下一行（10）。
- 0 标志着字符串的结束。

```
msg byte 'Hello, Assembly !',13,10,0
```

3. 操作数和参数

- 处理器指令的操作数：表示参与操作的对象
 - 具体的常量
 - 保存在寄存器的数据
 - 保存在存储器中的变量
 - 逗号前常是目的操作数，逗号后常是源操作数
- 伪指令的参数：
 - 常量、变量名、表达式等
 - 可以有多个，参数之间用逗号分隔

```
mov eax,offset msg
```

```
msg byte 'Hello, Assembly !',13,10,0
```

4. 注释和分隔符

- 语句中分号后的内容是注释
 - 对指令或程序进行说明
 - 汇编程序不对它们做任何处理
 - 注释利于阅读，应养成书写注释的好习惯
- 语句的4个组成部分要用分隔符分开
 - 标号后的冒号
 - 注释前的分号
 - 操作数间和参数间的逗号
 - 分隔其他部分采用一个或多个空格或制表符
 - 注释使用英文或中文均可

分隔符都是英文标点

良好的语句格式有利于编程

X86源程序框架

```
;eg0000.asm in Windows Console
include irvine32.inc
常量1 equ xxxx;
常量2 equ xxxx;
.data
    org XX
    .....
.code
main proc
    .....
    exit
main endp
    .....
end main
```

;包含32位输入输出文件
;根据需要定义常量，**可以没有**

;定义数据段
; **指定变量的存储偏移量，可以没有**
;数据定义（数据待填）
;定义代码段
;主过程开始
;主程序（指令待填）
;程序正常执行结束
;主过程结束
;子过程（指令待填）
;汇编结束

;通常从主过程的第一条指令开始执行

X86源程序框架

——包含伪指令INCLUDE

1. 将常用的常量定义、过程说明、共享的子程序库等内容进行声明（相当于C和C++语言中，包含头文件的作用）
2. **IRVINE32.INC是教学用的开发工具**的包含文件
 - 对用到的一些库函数做了原型声明
 - 包含了SmallWin.inc

X86源程序框架

——段的简化定义

1. 数据段定义伪指令 **.DATA** 创建一个数据段
2. 代码段定义伪指令 **.CODE** 创建一个代码段
3. 堆栈段由Windows维护，用户不必设置
4. 程序开始执行的位置
 - 应用一个标号（例如：START）或主过程main
 - 汇编结束END指令的参数
5. 应用程序执行结束
 - 语句 **“EXIT”** 将控制权交还操作系统
6. 源程序汇编结束END语句

执行结束 ≠ 汇编结束

X86源程序框架

——〔例〕 Hello, world!信息显示程序

1. 在数据段给出这个字符串形式的信息：

;数据段

```
msg    byte 'Hello, world!',13,10,0
```

;定义要显示的字符串

2. 在代码段编写显示字符串的程序：

;代码段

```
mov edx,offset msg
```

;edx指定字符串的偏移地址

```
call writestring
```

;调用I/O子程序显示信息

源程序框架

——X86汇编语言源程序

```
; include irvine32.inc
.data                      ;数据段
msg      byte 'Hello, world!',13,10,0
.code                      ;代码段
main proc                 ;主过程定义
    mov     edx,offset msg
    call    writestring
    exit    ;程序正常执行结束
main endp
end main                 ;汇编结束
```

Hello, world !

源程序框架

——C源程序

```
#include <stdio.h>
int main()
{
    printf("Hello, world !\n");
    exit(0);
}
```

运行结果

Hello, world !

源程序框架

---ARM源程序

```
.text
.global tart1
tart1:
    mov x0,#0
    ldr x1,=msg
    mov x2,len
    mov x8,64
    svc #0

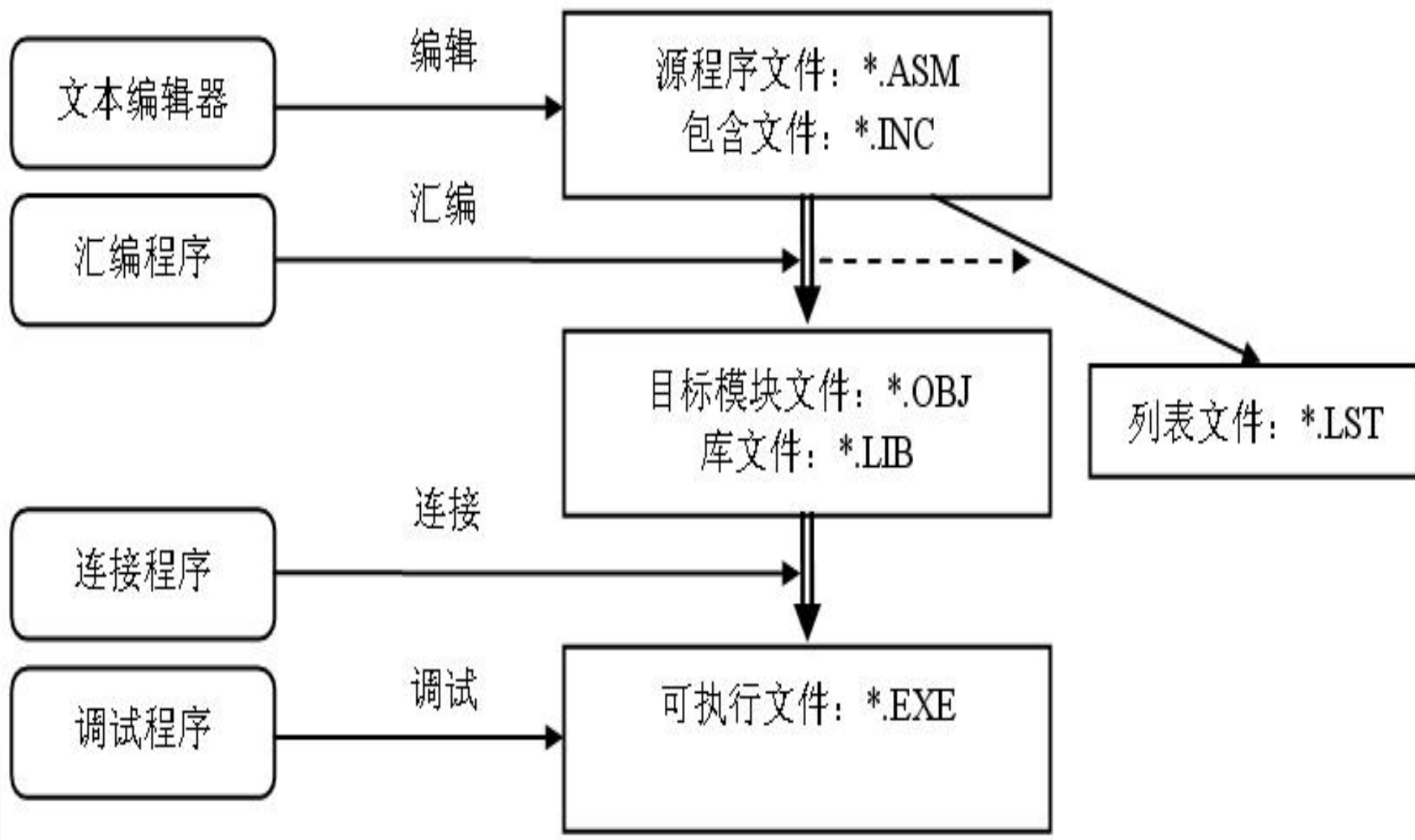
    mov x0,123
    mov x8,93
    svc #0

.data
msg:
    .ascii "Hello World!\n"
len=.-msg
```

运行结果

Hello world !

1.3.4 开发过程



2. 源程序的编辑

- 源程序文件是无格式文件、纯文本类型
- 以ASM为扩展名，可使用任何文本编辑器
 - Windows的记事本Notepad
 - 其他程序开发工具中的编辑环境
 - 专注于源程序编写的编辑软件，例如UltraEdit32
- 本书配套开发软件MASM主目录提供
 - Notepad2.exe程序

本书源程序文件的命名规则

- **EG**=例题，**EX**=习题
- 前**2**位数字=章号，后**2**位数字=序号

编译脚本

```
REM make32.bat - Batch file for assembling/linking 32-bit Assembly programs  
REM Revised: 11/15/01
```

```
@echo off  
cls
```

```
REM The following three lines can be customized for your system:
```

```
REM *****BEGIN customize
```

```
SET PATH=F:\fzuASM\bin
```

```
SET INCLUDE=F:\fzuASM\INCLUDE
```

```
SET LIB=F:\fzuASM\LIB
```

```
REM *****END customize
```

```
ML -Zi -c -Fl -coff %1.asm
```

```
if errorlevel 1 goto terminate
```

```
REM add the /MAP option for a map file in the link command.
```

```
LINK32 %1.obj irvine32.lib kernel32.lib /SUBSYSTEM:CONSOLE /DEBUG
```

```
if errorLevel 1 goto terminate
```

```
dir %1.*
```

```
:terminate
```

```
pause
```

3. 源程序

- 生成目标模块文件（.OBJ）
- MASM 6. x的汇编程序是ML. EXE

`BIN\ML /c /coff eg0101.asm`

- 参数“/c”（小写字母）实现源程序的汇编
- 参数“/coff”（小写字母）表示生成COFF格式的目标模块文件

温馨提示：ML.EXE的参数区别大小写

温馨提示：将源文件放在MASM目录

在 MASM 的汇编器 ML.EXE 中，命令行参数的写法有两种等价形式：
传统的 DOS 风格：使用斜杠 / 开头，如 /c、/coff
兼容的 Unix 风格：使用减号 - 开头，如 -c、-coff。
这两种写法在功能上是完全等价的，ML.EXE 都能正确识别。

4. 目标文件的连接

- 把一个或多个目标文件和库文件合成一个可执行文件（.EXE）
- 32位连接程序被更名为LINK32.EXE

`BIN\LINK32 /subsystem:console eg0101.obj`

- “/subsystem:console” 表示生成Windows控制台（Console）环境的可执行文件
- “/subsystem:windows” 生成Windows图形窗口的可执行文件

温馨提示： 使用批处理文件方便操作

温馨提示： 汇编程序只指出语法错误

5. 可执行文件的运行

- 进入控制台（或模拟DOS）环境
- 在命令行提示符下输入文件名（可以省略扩展名）、按下回车键运行

eg0101.exe

- 运行错误，就需要
 - 静态排错：阅读分析源程序
 - 动态排错：利用调试程序

温馨提示：不要在Windows下双击运行

6. 列表文件

- 列表文件（.LST）含有源程序和目标代码

ML /c /coff /F1 eg0101.asm

- “/F1” 创建列表文件
（大写F、小写1，不是数字1）

- 列表文件有两部分内容

- 第一部分：源程序及其代码
- 第二部分：各种标识符

- 错误Error：比较严重的语法错误
- 警告Warning：不太关键的语法错误

第一章课后作业

1.1 简答题 (1、3、6、7、8)

1.2 判断题 (1、6、7、8、9)

1.3 填空题 (3、4、5、7、10)

1.9、1.13、1.15、1.16

